

### Algorithms for linear programming

This chapter studies the simplex algorithm. This algorithm, when implemented carefully, often solves general linear programs quickly in practice. With some carefully contrived inputs, however, the simplex algorithm can require exponential time. The first polynomial-time algorithm for linear programming was the **ellipsoid algorithm**, which runs slowly in practice. A second class of polynomial-time algorithms are known as **interior-point methods**. In contrast to the simplex algorithm, which moves along the exterior of the feasible region and maintains a feasible solution that is a vertex of the simplex at each iteration, these algorithms move through the interior of the feasible region. The intermediate solutions, while feasible, are not necessarily vertices of the simplex, but the final solution is a vertex. For large inputs, interior-point algorithms can run as fast as, and sometimes faster than, the simplex algorithm. The chapter notes point you to more information about these algorithms.

If we add to a linear program the additional requirement that all variables take on integer values, we have an **integer linear program**. Exercise 34.5-3 asks you to show that just finding a feasible solution to this problem is NP-hard; since no polynomial-time algorithms are known for any NP-hard problems, there is no known polynomial-time algorithm for integer linear programming. In contrast, we can solve a general linear-programming problem in polynomial time.

In this chapter, if we have a linear program with variables  $x = (x_1, x_2, \dots, x_n)$  and wish to refer to a particular setting of the variables, we shall use the notation  $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ .

---

## 29.1 Standard and slack forms

This section describes two formats, standard form and slack form, that are useful when we specify and work with linear programs. In standard form, all the constraints are inequalities, whereas in slack form, all constraints are equalities (except for those that require the variables to be nonnegative).

### Standard form

In **standard form**, we are given  $n$  real numbers  $c_1, c_2, \dots, c_n$ ;  $m$  real numbers  $b_1, b_2, \dots, b_m$ ; and  $mn$  real numbers  $a_{ij}$  for  $i = 1, 2, \dots, m$  and  $j = 1, 2, \dots, n$ . We wish to find  $n$  real numbers  $x_1, x_2, \dots, x_n$  that

$$\text{maximize} \quad \sum_{j=1}^n c_j x_j \quad (29.16)$$

subject to

$$\sum_{j=1}^n a_{ij} x_j \leq b_i \quad \text{for } i = 1, 2, \dots, m \quad (29.17)$$

$$x_j \geq 0 \quad \text{for } j = 1, 2, \dots, n. \quad (29.18)$$

Generalizing the terminology we introduced for the two-variable linear program, we call expression (29.16) the **objective function** and the  $n + m$  inequalities in lines (29.17) and (29.18) the **constraints**. The  $n$  constraints in line (29.18) are the **nonnegativity constraints**. An arbitrary linear program need not have nonnegativity constraints, but standard form requires them. Sometimes we find it convenient to express a linear program in a more compact form. If we create an  $m \times n$  matrix  $A = (a_{ij})$ , an  $m$ -vector  $b = (b_i)$ , an  $n$ -vector  $c = (c_j)$ , and an  $n$ -vector  $x = (x_j)$ , then we can rewrite the linear program defined in (29.16)–(29.18) as

$$\text{maximize} \quad c^T x \quad (29.19)$$

subject to

$$Ax \leq b \quad (29.20)$$

$$x \geq 0. \quad (29.21)$$

In line (29.19),  $c^T x$  is the inner product of two vectors. In inequality (29.20),  $Ax$  is a matrix-vector product, and in inequality (29.21),  $x \geq 0$  means that each entry of the vector  $x$  must be nonnegative. We see that we can specify a linear program in standard form by a tuple  $(A, b, c)$ , and we shall adopt the convention that  $A$ ,  $b$ , and  $c$  always have the dimensions given above.

We now introduce terminology to describe solutions to linear programs. We used some of this terminology in the earlier example of a two-variable linear program. We call a setting of the variables  $\bar{x}$  that satisfies all the constraints a **feasible solution**, whereas a setting of the variables  $\bar{x}$  that fails to satisfy at least one constraint is an **infeasible solution**. We say that a solution  $\bar{x}$  has **objective value**  $c^T \bar{x}$ . A feasible solution  $\bar{x}$  whose objective value is maximum over all feasible solutions is an **optimal solution**, and we call its objective value  $c^T \bar{x}$  the **optimal objective value**. If a linear program has no feasible solutions, we say that the linear program is **infeasible**; otherwise it is **feasible**. If a linear program has some feasible solutions but does not have a finite optimal objective value, we say that the linear program is **unbounded**. Exercise 29.1-9 asks you to show that a linear program can have a finite optimal objective value even if the feasible region is not bounded.

### Converting linear programs into standard form

It is always possible to convert a linear program, given as minimizing or maximizing a linear function subject to linear constraints, into standard form. A linear program might not be in standard form for any of four possible reasons:

1. The objective function might be a minimization rather than a maximization.
2. There might be variables without nonnegativity constraints.
3. There might be **equality constraints**, which have an equal sign rather than a less-than-or-equal-to sign.
4. There might be **inequality constraints**, but instead of having a less-than-or-equal-to sign, they have a greater-than-or-equal-to sign.

When converting one linear program  $L$  into another linear program  $L'$ , we would like the property that an optimal solution to  $L'$  yields an optimal solution to  $L$ . To capture this idea, we say that two maximization linear programs  $L$  and  $L'$  are **equivalent** if for each feasible solution  $\bar{x}$  to  $L$  with objective value  $z$ , there is a corresponding feasible solution  $\bar{x}'$  to  $L'$  with objective value  $z$ , and for each feasible solution  $\bar{x}'$  to  $L'$  with objective value  $z$ , there is a corresponding feasible solution  $\bar{x}$  to  $L$  with objective value  $z$ . (This definition does not imply a one-to-one correspondence between feasible solutions.) A minimization linear program  $L$  and a maximization linear program  $L'$  are equivalent if for each feasible solution  $\bar{x}$  to  $L$  with objective value  $z$ , there is a corresponding feasible solution  $\bar{x}'$  to  $L'$  with objective value  $-z$ , and for each feasible solution  $\bar{x}'$  to  $L'$  with objective value  $z$ , there is a corresponding feasible solution  $\bar{x}$  to  $L$  with objective value  $-z$ .

We now show how to remove, one by one, each of the possible problems in the list above. After removing each one, we shall argue that the new linear program is equivalent to the old one.

To convert a minimization linear program  $L$  into an equivalent maximization linear program  $L'$ , we simply negate the coefficients in the objective function. Since  $L$  and  $L'$  have identical sets of feasible solutions and, for any feasible solution, the objective value in  $L$  is the negative of the objective value in  $L'$ , these two linear programs are equivalent. For example, if we have the linear program

$$\begin{array}{ll} \text{minimize} & -2x_1 + 3x_2 \\ \text{subject to} & \\ & x_1 + x_2 = 7 \\ & x_1 - 2x_2 \leq 4 \\ & x_1 \geq 0, \end{array}$$

and we negate the coefficients of the objective function, we obtain

$$\begin{array}{llll}
\text{maximize} & 2x_1 & - & 3x_2 \\
\text{subject to} & & & \\
& x_1 & + & x_2 = 7 \\
& x_1 & - & 2x_2 \leq 4 \\
& x_1 & & \geq 0 .
\end{array}$$

Next, we show how to convert a linear program in which some of the variables do not have nonnegativity constraints into one in which each variable has a nonnegativity constraint. Suppose that some variable  $x_j$  does not have a nonnegativity constraint. Then, we replace each occurrence of  $x_j$  by  $x'_j - x''_j$ , and add the nonnegativity constraints  $x'_j \geq 0$  and  $x''_j \geq 0$ . Thus, if the objective function has a term  $c_j x_j$ , we replace it by  $c_j x'_j - c_j x''_j$ , and if constraint  $i$  has a term  $a_{ij} x_j$ , we replace it by  $a_{ij} x'_j - a_{ij} x''_j$ . Any feasible solution  $\hat{x}$  to the new linear program corresponds to a feasible solution  $\bar{x}$  to the original linear program with  $\bar{x}_j = \hat{x}'_j - \hat{x}''_j$  and with the same objective value. Also, any feasible solution  $\bar{x}$  to the original linear program corresponds to a feasible solution  $\hat{x}$  to the new linear program with  $\hat{x}'_j = \bar{x}_j$  and  $\hat{x}''_j = 0$  if  $\bar{x}_j \geq 0$ , or with  $\hat{x}'_j = -\bar{x}_j$  and  $\hat{x}''_j = 0$  if  $\bar{x}_j < 0$ . The two linear programs have the same objective value regardless of the sign of  $\bar{x}_j$ . Thus, the two linear programs are equivalent. We apply this conversion scheme to each variable that does not have a nonnegativity constraint to yield an equivalent linear program in which all variables have nonnegativity constraints.

Continuing the example, we want to ensure that each variable has a corresponding nonnegativity constraint. Variable  $x_1$  has such a constraint, but variable  $x_2$  does not. Therefore, we replace  $x_2$  by two variables  $x'_2$  and  $x''_2$ , and we modify the linear program to obtain

$$\begin{array}{llllll}
\text{maximize} & 2x_1 & - & 3x'_2 & + & 3x''_2 \\
\text{subject to} & & & & & \\
& x_1 & + & x'_2 & - & x''_2 = 7 \\
& x_1 & - & 2x'_2 & + & 2x''_2 \leq 4 \\
& x_1, x'_2, x''_2 & & & & \geq 0 .
\end{array} \tag{29.22}$$

Next, we convert equality constraints into inequality constraints. Suppose that a linear program has an equality constraint  $f(x_1, x_2, \dots, x_n) = b$ . Since  $x = y$  if and only if both  $x \geq y$  and  $x \leq y$ , we can replace this equality constraint by the pair of inequality constraints  $f(x_1, x_2, \dots, x_n) \leq b$  and  $f(x_1, x_2, \dots, x_n) \geq b$ . Repeating this conversion for each equality constraint yields a linear program in which all constraints are inequalities.

Finally, we can convert the greater-than-or-equal-to constraints to less-than-or-equal-to constraints by multiplying these constraints through by  $-1$ . That is, any inequality of the form

$$\sum_{j=1}^n a_{ij}x_j \geq b_i$$

is equivalent to

$$\sum_{j=1}^n -a_{ij}x_j \leq -b_i .$$

Thus, by replacing each coefficient  $a_{ij}$  by  $-a_{ij}$  and each value  $b_i$  by  $-b_i$ , we obtain an equivalent less-than-or-equal-to constraint.

Finishing our example, we replace the equality in constraint (29.22) by two inequalities, obtaining

$$\begin{aligned} &\text{maximize} && 2x_1 &-& 3x'_2 &+& 3x''_2 \\ &\text{subject to} && x_1 &+& x'_2 &-& x''_2 &\leq 7 \\ &&& x_1 &+& x'_2 &-& x''_2 &\geq 7 \\ &&& x_1 &-& 2x'_2 &+& 2x''_2 &\leq 4 \\ &&& x_1, x'_2, x''_2 &&&&&\geq 0 . \end{aligned} \tag{29.23}$$

Finally, we negate constraint (29.23). For consistency in variable names, we rename  $x'_2$  to  $x_2$  and  $x''_2$  to  $x_3$ , obtaining the standard form

$$\text{maximize} \quad 2x_1 - 3x_2 + 3x_3 \tag{29.24}$$

subject to

$$x_1 + x_2 - x_3 \leq 7 \tag{29.25}$$

$$-x_1 - x_2 + x_3 \leq -7 \tag{29.26}$$

$$x_1 - 2x_2 + 2x_3 \leq 4 \tag{29.27}$$

$$x_1, x_2, x_3 \geq 0 . \tag{29.28}$$

### Converting linear programs into slack form

To efficiently solve a linear program with the simplex algorithm, we prefer to express it in a form in which some of the constraints are equality constraints. More precisely, we shall convert it into a form in which the nonnegativity constraints are the only inequality constraints, and the remaining constraints are equalities. Let

$$\sum_{j=1}^n a_{ij}x_j \leq b_i \tag{29.29}$$

be an inequality constraint. We introduce a new variable  $s$  and rewrite inequality (29.29) as the two constraints

$$s = b_i - \sum_{j=1}^n a_{ij} x_j, \quad (29.30)$$

$$s \geq 0. \quad (29.31)$$

We call  $s$  a **slack variable** because it measures the **slack**, or difference, between the left-hand and right-hand sides of equation (29.29). (We shall soon see why we find it convenient to write the constraint with only the slack variable on the left-hand side.) Because inequality (29.29) is true if and only if both equation (29.30) and inequality (29.31) are true, we can convert each inequality constraint of a linear program in this way to obtain an equivalent linear program in which the only inequality constraints are the nonnegativity constraints. When converting from standard to slack form, we shall use  $x_{n+i}$  (instead of  $s$ ) to denote the slack variable associated with the  $i$ th inequality. The  $i$ th constraint is therefore

$$x_{n+i} = b_i - \sum_{j=1}^n a_{ij} x_j, \quad (29.32)$$

along with the nonnegativity constraint  $x_{n+i} \geq 0$ .

By converting each constraint of a linear program in standard form, we obtain a linear program in a different form. For example, for the linear program described in (29.24)–(29.28), we introduce slack variables  $x_4$ ,  $x_5$ , and  $x_6$ , obtaining

$$\text{maximize} \quad 2x_1 - 3x_2 + 3x_3 \quad (29.33)$$

subject to

$$x_4 = 7 - x_1 - x_2 + x_3 \quad (29.34)$$

$$x_5 = -7 + x_1 + x_2 - x_3 \quad (29.35)$$

$$x_6 = 4 - x_1 + 2x_2 - 2x_3 \quad (29.36)$$

$$x_1, x_2, x_3, x_4, x_5, x_6 \geq 0. \quad (29.37)$$

In this linear program, all the constraints except for the nonnegativity constraints are equalities, and each variable is subject to a nonnegativity constraint. We write each equality constraint with one of the variables on the left-hand side of the equality and all others on the right-hand side. Furthermore, each equation has the same set of variables on the right-hand side, and these variables are also the only ones that appear in the objective function. We call the variables on the left-hand side of the equalities **basic variables** and those on the right-hand side **nonbasic variables**.

For linear programs that satisfy these conditions, we shall sometimes omit the words “maximize” and “subject to,” as well as the explicit nonnegativity constraints. We shall also use the variable  $z$  to denote the value of the objective func-

tion. We call the resulting format **slack form**. If we write the linear program given in (29.33)–(29.37) in slack form, we obtain

$$z = 2x_1 - 3x_2 + 3x_3 \quad (29.38)$$

$$x_4 = 7 - x_1 - x_2 + x_3 \quad (29.39)$$

$$x_5 = -7 + x_1 + x_2 - x_3 \quad (29.40)$$

$$x_6 = 4 - x_1 + 2x_2 - 2x_3. \quad (29.41)$$

As with standard form, we find it convenient to have a more concise notation for describing a slack form. As we shall see in Section 29.3, the sets of basic and nonbasic variables will change as the simplex algorithm runs. We use  $N$  to denote the set of indices of the nonbasic variables and  $B$  to denote the set of indices of the basic variables. We always have that  $|N| = n$ ,  $|B| = m$ , and  $N \cup B = \{1, 2, \dots, n + m\}$ . The equations are indexed by the entries of  $B$ , and the variables on the right-hand sides are indexed by the entries of  $N$ . As in standard form, we use  $b_i$ ,  $c_j$ , and  $a_{ij}$  to denote constant terms and coefficients. We also use  $v$  to denote an optional constant term in the objective function. (We shall see a little later that including the constant term in the objective function makes it easy to determine the value of the objective function.) Thus we can concisely define a slack form by a tuple  $(N, B, A, b, c, v)$ , denoting the slack form

$$z = v + \sum_{j \in N} c_j x_j \quad (29.42)$$

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j \quad \text{for } i \in B, \quad (29.43)$$

in which all variables  $x$  are constrained to be nonnegative. Because we subtract the sum  $\sum_{j \in N} a_{ij} x_j$  in (29.43), the values  $a_{ij}$  are actually the negatives of the coefficients as they “appear” in the slack form.

For example, in the slack form

$$\begin{aligned} z &= 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3} \\ x_1 &= 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3} \\ x_2 &= 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3} \\ x_4 &= 18 - \frac{x_3}{2} + \frac{x_5}{2}, \end{aligned}$$

we have  $B = \{1, 2, 4\}$ ,  $N = \{3, 5, 6\}$ ,

$$A = \begin{pmatrix} a_{13} & a_{15} & a_{16} \\ a_{23} & a_{25} & a_{26} \\ a_{43} & a_{45} & a_{46} \end{pmatrix} = \begin{pmatrix} -1/6 & -1/6 & 1/3 \\ 8/3 & 2/3 & -1/3 \\ 1/2 & -1/2 & 0 \end{pmatrix},$$

$$b = \begin{pmatrix} b_1 \\ b_2 \\ b_4 \end{pmatrix} = \begin{pmatrix} 8 \\ 4 \\ 18 \end{pmatrix},$$

$c = (c_3 \ c_5 \ c_6)^T = (-1/6 \ -1/6 \ -2/3)^T$ , and  $v = 28$ . Note that the indices into  $A$ ,  $b$ , and  $c$  are not necessarily sets of contiguous integers; they depend on the index sets  $B$  and  $N$ . As an example of the entries of  $A$  being the negatives of the coefficients as they appear in the slack form, observe that the equation for  $x_1$  includes the term  $x_3/6$ , yet the coefficient  $a_{13}$  is actually  $-1/6$  rather than  $+1/6$ .

### Exercises

#### 29.1-1

If we express the linear program in (29.24)–(29.28) in the compact notation of (29.19)–(29.21), what are  $n$ ,  $m$ ,  $A$ ,  $b$ , and  $c$ ?

#### 29.1-2

Give three feasible solutions to the linear program in (29.24)–(29.28). What is the objective value of each one?

#### 29.1-3

For the slack form in (29.38)–(29.41), what are  $N$ ,  $B$ ,  $A$ ,  $b$ ,  $c$ , and  $v$ ?

#### 29.1-4

Convert the following linear program into standard form:

$$\begin{array}{llllll} \text{minimize} & 2x_1 & + & 7x_2 & + & x_3 \\ \text{subject to} & & & & & \\ & x_1 & & & - & x_3 & = & 7 \\ & 3x_1 & + & x_2 & & & \geq & 24 \\ & & & x_2 & & & \geq & 0 \\ & & & & & x_3 & \leq & 0 \end{array}.$$



**29.1-5**

Convert the following linear program into slack form:

$$\begin{array}{llllll}
 \text{maximize} & 2x_1 & & - & 6x_3 & \\
 \text{subject to} & & & & & \\
 & x_1 & + & x_2 & - & x_3 \leq 7 \\
 & 3x_1 & - & x_2 & & \geq 8 \\
 & -x_1 & + & 2x_2 & + & 2x_3 \geq 0 \\
 & x_1, x_2, x_3 & & & & \geq 0 .
 \end{array}$$

What are the basic and nonbasic variables?

**29.1-6**

Show that the following linear program is infeasible:

$$\begin{array}{llllll}
 \text{maximize} & 3x_1 & - & 2x_2 & & \\
 \text{subject to} & & & & & \\
 & x_1 & + & x_2 & \leq & 2 \\
 & -2x_1 & - & 2x_2 & \leq & -10 \\
 & x_1, x_2 & & & \geq & 0 .
 \end{array}$$

**29.1-7**

Show that the following linear program is unbounded:

$$\begin{array}{llllll}
 \text{maximize} & x_1 & - & x_2 & & \\
 \text{subject to} & & & & & \\
 & -2x_1 & + & x_2 & \leq & -1 \\
 & -x_1 & - & 2x_2 & \leq & -2 \\
 & x_1, x_2 & & & \geq & 0 .
 \end{array}$$

**29.1-8**

Suppose that we have a general linear program with  $n$  variables and  $m$  constraints, and suppose that we convert it into standard form. Give an upper bound on the number of variables and constraints in the resulting linear program.

**29.1-9**

Give an example of a linear program for which the feasible region is not bounded, but the optimal objective value is finite.

---

## 29.2 Formulating problems as linear programs

Although we shall focus on the simplex algorithm in this chapter, it is also important to be able to recognize when we can formulate a problem as a linear program. Once we cast a problem as a polynomial-sized linear program, we can solve it in polynomial time by the ellipsoid algorithm or interior-point methods. Several linear-programming software packages can solve problems efficiently, so that once the problem is in the form of a linear program, such a package can solve it.

We shall look at several concrete examples of linear-programming problems. We start with two problems that we have already studied: the single-source shortest-paths problem (see Chapter 24) and the maximum-flow problem (see Chapter 26). We then describe the minimum-cost-flow problem. Although the minimum-cost-flow problem has a polynomial-time algorithm that is not based on linear programming, we won't describe the algorithm. Finally, we describe the multicommodity-flow problem, for which the only known polynomial-time algorithm is based on linear programming.

When we solved graph problems in Part VI, we used attribute notation, such as  $v.d$  and  $(u, v).f$ . Linear programs typically use subscripted variables rather than objects with attached attributes, however. Therefore, when we express variables in linear programs, we shall indicate vertices and edges through subscripts. For example, we denote the shortest-path weight for vertex  $v$  not by  $v.d$  but by  $d_v$ . Similarly, we denote the flow from vertex  $u$  to vertex  $v$  not by  $(u, v).f$  but by  $f_{uv}$ . For quantities that are given as inputs to problems, such as edge weights or capacities, we shall continue to use notations such as  $w(u, v)$  and  $c(u, v)$ .

### Shortest paths

We can formulate the single-source shortest-paths problem as a linear program. In this section, we shall focus on how to formulate the single-pair shortest-path problem, leaving the extension to the more general single-source shortest-paths problem as Exercise 29.2-3.

In the single-pair shortest-path problem, we are given a weighted, directed graph  $G = (V, E)$ , with weight function  $w : E \rightarrow \mathbb{R}$  mapping edges to real-valued weights, a source vertex  $s$ , and destination vertex  $t$ . We wish to compute the value  $d_t$ , which is the weight of a shortest path from  $s$  to  $t$ . To express this problem as a linear program, we need to determine a set of variables and constraints that define when we have a shortest path from  $s$  to  $t$ . Fortunately, the Bellman-Ford algorithm does exactly this. When the Bellman-Ford algorithm terminates, it has computed, for each vertex  $v$ , a value  $d_v$  (using subscript notation here rather than attribute notation) such that for each edge  $(u, v) \in E$ , we have  $d_v \leq d_u + w(u, v)$ .

The source vertex initially receives a value  $d_s = 0$ , which never changes. Thus we obtain the following linear program to compute the shortest-path weight from  $s$  to  $t$ :

$$\text{maximize } d_t \quad (29.44)$$

subject to

$$d_v \leq d_u + w(u, v) \quad \text{for each edge } (u, v) \in E, \quad (29.45)$$

$$d_s = 0. \quad (29.46)$$

You might be surprised that this linear program maximizes an objective function when it is supposed to compute shortest paths. We do not want to minimize the objective function, since then setting  $\bar{d}_v = 0$  for all  $v \in V$  would yield an optimal solution to the linear program without solving the shortest-paths problem. We maximize because an optimal solution to the shortest-paths problem sets each  $\bar{d}_v$  to  $\min_{u:(u,v) \in E} \{\bar{d}_u + w(u, v)\}$ , so that  $\bar{d}_v$  is the largest value that is less than or equal to all of the values in the set  $\{\bar{d}_u + w(u, v)\}$ . We want to maximize  $d_v$  for all vertices  $v$  on a shortest path from  $s$  to  $t$  subject to these constraints on all vertices  $v$ , and maximizing  $d_t$  achieves this goal.

This linear program has  $|V|$  variables  $d_v$ , one for each vertex  $v \in V$ . It also has  $|E| + 1$  constraints: one for each edge, plus the additional constraint that the source vertex's shortest-path weight always has the value 0.

### Maximum flow

Next, we express the maximum-flow problem as a linear program. Recall that we are given a directed graph  $G = (V, E)$  in which each edge  $(u, v) \in E$  has a nonnegative capacity  $c(u, v) \geq 0$ , and two distinguished vertices: a source  $s$  and a sink  $t$ . As defined in Section 26.1, a flow is a nonnegative real-valued function  $f : V \times V \rightarrow \mathbb{R}$  that satisfies the capacity constraint and flow conservation. A maximum flow is a flow that satisfies these constraints and maximizes the flow value, which is the total flow coming out of the source minus the total flow into the source. A flow, therefore, satisfies linear constraints, and the value of a flow is a linear function. Recalling also that we assume that  $c(u, v) = 0$  if  $(u, v) \notin E$  and that there are no antiparallel edges, we can express the maximum-flow problem as a linear program:

$$\text{maximize } \sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs} \quad (29.47)$$

subject to

$$f_{uv} \leq c(u, v) \quad \text{for each } u, v \in V, \quad (29.48)$$

$$\sum_{v \in V} f_{vu} = \sum_{v \in V} f_{uv} \quad \text{for each } u \in V - \{s, t\}, \quad (29.49)$$

$$f_{uv} \geq 0 \quad \text{for each } u, v \in V. \quad (29.50)$$

This linear program has  $|V|^2$  variables, corresponding to the flow between each pair of vertices, and it has  $2|V|^2 + |V| - 2$  constraints.

It is usually more efficient to solve a smaller-sized linear program. The linear program in (29.47)–(29.50) has, for ease of notation, a flow and capacity of 0 for each pair of vertices  $u, v$  with  $(u, v) \notin E$ . It would be more efficient to rewrite the linear program so that it has  $O(V + E)$  constraints. Exercise 29.2-5 asks you to do so.

### Minimum-cost flow

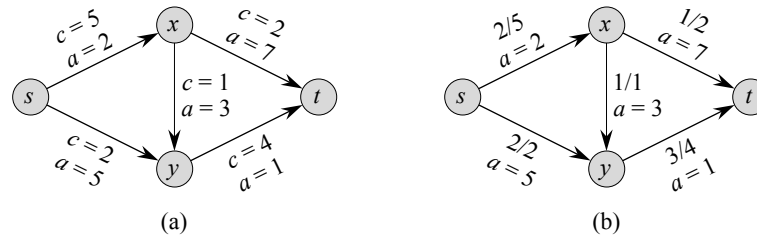
In this section, we have used linear programming to solve problems for which we already knew efficient algorithms. In fact, an efficient algorithm designed specifically for a problem, such as Dijkstra's algorithm for the single-source shortest-paths problem, or the push-relabel method for maximum flow, will often be more efficient than linear programming, both in theory and in practice.

The real power of linear programming comes from the ability to solve new problems. Recall the problem faced by the politician in the beginning of this chapter. The problem of obtaining a sufficient number of votes, while not spending too much money, is not solved by any of the algorithms that we have studied in this book, yet we can solve it by linear programming. Books abound with such real-world problems that linear programming can solve. Linear programming is also particularly useful for solving variants of problems for which we may not already know of an efficient algorithm.

Consider, for example, the following generalization of the maximum-flow problem. Suppose that, in addition to a capacity  $c(u, v)$  for each edge  $(u, v)$ , we are given a real-valued cost  $a(u, v)$ . As in the maximum-flow problem, we assume that  $c(u, v) = 0$  if  $(u, v) \notin E$ , and that there are no antiparallel edges. If we send  $f_{uv}$  units of flow over edge  $(u, v)$ , we incur a cost of  $a(u, v)f_{uv}$ . We are also given a flow demand  $d$ . We wish to send  $d$  units of flow from  $s$  to  $t$  while minimizing the total cost  $\sum_{(u,v) \in E} a(u, v)f_{uv}$  incurred by the flow. This problem is known as the **minimum-cost-flow problem**.

Figure 29.3(a) shows an example of the minimum-cost-flow problem. We wish to send 4 units of flow from  $s$  to  $t$  while incurring the minimum total cost. Any particular legal flow, that is, a function  $f$  satisfying constraints (29.48)–(29.50), incurs a total cost of  $\sum_{(u,v) \in E} a(u, v)f_{uv}$ . We wish to find the particular 4-unit flow that minimizes this cost. Figure 29.3(b) shows an optimal solution, with total cost  $\sum_{(u,v) \in E} a(u, v)f_{uv} = (2 \cdot 2) + (5 \cdot 2) + (3 \cdot 1) + (7 \cdot 1) + (1 \cdot 3) = 27$ .

There are polynomial-time algorithms specifically designed for the minimum-cost-flow problem, but they are beyond the scope of this book. We can, however, express the minimum-cost-flow problem as a linear program. The linear program looks similar to the one for the maximum-flow problem with the additional con-



**Figure 29.3** (a) An example of a minimum-cost-flow problem. We denote the capacities by  $c$  and the costs by  $a$ . Vertex  $s$  is the source and vertex  $t$  is the sink, and we wish to send 4 units of flow from  $s$  to  $t$ . (b) A solution to the minimum-cost flow problem in which 4 units of flow are sent from  $s$  to  $t$ . For each edge, the flow and capacity are written as flow/capacity.

straint that the value of the flow be exactly  $d$  units, and with the new objective function of minimizing the cost:

$$\text{minimize} \quad \sum_{(u,v) \in E} a(u,v) f_{uv} \quad (29.51)$$

subject to

$$\begin{aligned} f_{uv} &\leq c(u,v) && \text{for each } u, v \in V, \\ \sum_{v \in V} f_{vu} - \sum_{v \in V} f_{uv} &= 0 && \text{for each } u \in V - \{s, t\}, \\ \sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs} &= d, \\ f_{uv} &\geq 0 && \text{for each } u, v \in V. \end{aligned} \quad (29.52)$$

### Multicommodity flow

As a final example, we consider another flow problem. Suppose that the Lucky Puck company from Section 26.1 decides to diversify its product line and ship not only hockey pucks, but also hockey sticks and hockey helmets. Each piece of equipment is manufactured in its own factory, has its own warehouse, and must be shipped, each day, from factory to warehouse. The sticks are manufactured in Vancouver and must be shipped to Saskatoon, and the helmets are manufactured in Edmonton and must be shipped to Regina. The capacity of the shipping network does not change, however, and the different items, or **commodities**, must share the same network.

This example is an instance of a **multicommodity-flow problem**. In this problem, we are again given a directed graph  $G = (V, E)$  in which each edge  $(u, v) \in E$  has a nonnegative capacity  $c(u, v) \geq 0$ . As in the maximum-flow problem, we implicitly assume that  $c(u, v) = 0$  for  $(u, v) \notin E$ , and that the graph has no antipar-

allel edges. In addition, we are given  $k$  different commodities,  $K_1, K_2, \dots, K_k$ , where we specify commodity  $i$  by the triple  $K_i = (s_i, t_i, d_i)$ . Here, vertex  $s_i$  is the source of commodity  $i$ , vertex  $t_i$  is the sink of commodity  $i$ , and  $d_i$  is the demand for commodity  $i$ , which is the desired flow value for the commodity from  $s_i$  to  $t_i$ . We define a flow for commodity  $i$ , denoted by  $f_i$ , (so that  $f_{iuv}$  is the flow of commodity  $i$  from vertex  $u$  to vertex  $v$ ) to be a real-valued function that satisfies the flow-conservation and capacity constraints. We now define  $f_{uv}$ , the **aggregate flow**, to be the sum of the various commodity flows, so that  $f_{uv} = \sum_{i=1}^k f_{iuv}$ . The aggregate flow on edge  $(u, v)$  must be no more than the capacity of edge  $(u, v)$ . We are not trying to minimize any objective function in this problem; we need only determine whether such a flow exists. Thus, we write a linear program with a “null” objective function:

$$\begin{array}{ll}
 \text{minimize} & 0 \\
 \text{subject to} & \\
 & \sum_{i=1}^k f_{iuv} \leq c(u, v) \quad \text{for each } u, v \in V, \\
 & \sum_{v \in V} f_{iuv} - \sum_{v \in V} f_{ivu} = 0 \quad \text{for each } i = 1, 2, \dots, k \text{ and} \\
 & \quad \text{for each } u \in V - \{s_i, t_i\}, \\
 & \sum_{v \in V} f_{i, s_i, v} - \sum_{v \in V} f_{i, v, s_i} = d_i \quad \text{for each } i = 1, 2, \dots, k, \\
 & f_{iuv} \geq 0 \quad \text{for each } u, v \in V \text{ and} \\
 & \quad \text{for each } i = 1, 2, \dots, k.
 \end{array}$$

The only known polynomial-time algorithm for this problem expresses it as a linear program and then solves it with a polynomial-time linear-programming algorithm.

## Exercises

### 29.2-1

Put the single-pair shortest-path linear program from (29.44)–(29.46) into standard form.

### 29.2-2

Write out explicitly the linear program corresponding to finding the shortest path from node  $s$  to node  $y$  in Figure 24.2(a).

### 29.2-3

In the single-source shortest-paths problem, we want to find the shortest-path weights from a source vertex  $s$  to all vertices  $v \in V$ . Given a graph  $G$ , write a

linear program for which the solution has the property that  $d_v$  is the shortest-path weight from  $s$  to  $v$  for each vertex  $v \in V$ .

#### 29.2-4

Write out explicitly the linear program corresponding to finding the maximum flow in Figure 26.1(a).

#### 29.2-5

Rewrite the linear program for maximum flow (29.47)–(29.50) so that it uses only  $O(V + E)$  constraints.

#### 29.2-6

Write a linear program that, given a bipartite graph  $G = (V, E)$ , solves the maximum-bipartite-matching problem.

#### 29.2-7

In the **minimum-cost multicommodity-flow problem**, we are given directed graph  $G = (V, E)$  in which each edge  $(u, v) \in E$  has a nonnegative capacity  $c(u, v) \geq 0$  and a cost  $a(u, v)$ . As in the multicommodity-flow problem, we are given  $k$  different commodities,  $K_1, K_2, \dots, K_k$ , where we specify commodity  $i$  by the triple  $K_i = (s_i, t_i, d_i)$ . We define the flow  $f_i$  for commodity  $i$  and the aggregate flow  $f_{uv}$  on edge  $(u, v)$  as in the multicommodity-flow problem. A feasible flow is one in which the aggregate flow on each edge  $(u, v)$  is no more than the capacity of edge  $(u, v)$ . The cost of a flow is  $\sum_{u,v \in V} a(u, v) f_{uv}$ , and the goal is to find the feasible flow of minimum cost. Express this problem as a linear program.

---

### 29.3 The simplex algorithm

The simplex algorithm is the classical method for solving linear programs. In contrast to most of the other algorithms in this book, its running time is not polynomial in the worst case. It does yield insight into linear programs, however, and is often remarkably fast in practice.

In addition to having a geometric interpretation, described earlier in this chapter, the simplex algorithm bears some similarity to Gaussian elimination, discussed in Section 28.1. Gaussian elimination begins with a system of linear equalities whose solution is unknown. In each iteration, we rewrite this system in an equivalent form that has some additional structure. After some number of iterations, we have rewritten the system so that the solution is simple to obtain. The simplex algorithm proceeds in a similar manner, and we can view it as Gaussian elimination for inequalities.

We now describe the main idea behind an iteration of the simplex algorithm. Associated with each iteration will be a “basic solution” that we can easily obtain from the slack form of the linear program: set each nonbasic variable to 0 and compute the values of the basic variables from the equality constraints. An iteration converts one slack form into an equivalent slack form. The objective value of the associated basic feasible solution will be no less than that at the previous iteration, and usually greater. To achieve this increase in the objective value, we choose a nonbasic variable such that if we were to increase that variable’s value from 0, then the objective value would increase, too. The amount by which we can increase the variable is limited by the other constraints. In particular, we raise it until some basic variable becomes 0. We then rewrite the slack form, exchanging the roles of that basic variable and the chosen nonbasic variable. Although we have used a particular setting of the variables to guide the algorithm, and we shall use it in our proofs, the algorithm does not explicitly maintain this solution. It simply rewrites the linear program until an optimal solution becomes “obvious.”

### An example of the simplex algorithm

We begin with an extended example. Consider the following linear program in standard form:

$$\text{maximize } 3x_1 + x_2 + 2x_3 \quad (29.53)$$

subject to

$$x_1 + x_2 + 3x_3 \leq 30 \quad (29.54)$$

$$2x_1 + 2x_2 + 5x_3 \leq 24 \quad (29.55)$$

$$4x_1 + x_2 + 2x_3 \leq 36 \quad (29.56)$$

$$x_1, x_2, x_3 \geq 0. \quad (29.57)$$

In order to use the simplex algorithm, we must convert the linear program into slack form; we saw how to do so in Section 29.1. In addition to being an algebraic manipulation, slack is a useful algorithmic concept. Recalling from Section 29.1 that each variable has a corresponding nonnegativity constraint, we say that an equality constraint is *tight* for a particular setting of its nonbasic variables if they cause the constraint’s basic variable to become 0. Similarly, a setting of the nonbasic variables that would make a basic variable become negative *violates* that constraint. Thus, the slack variables explicitly maintain how far each constraint is from being tight, and so they help to determine how much we can increase values of nonbasic variables without violating any constraints.

Associating the slack variables  $x_4$ ,  $x_5$ , and  $x_6$  with inequalities (29.54)–(29.56), respectively, and putting the linear program into slack form, we obtain



$$z = 3x_1 + x_2 + 2x_3 \quad (29.58)$$

$$x_4 = 30 - x_1 - x_2 - 3x_3 \quad (29.59)$$

$$x_5 = 24 - 2x_1 - 2x_2 - 5x_3 \quad (29.60)$$

$$x_6 = 36 - 4x_1 - x_2 - 2x_3 . \quad (29.61)$$

The system of constraints (29.59)–(29.61) has 3 equations and 6 variables. Any setting of the variables  $x_1$ ,  $x_2$ , and  $x_3$  defines values for  $x_4$ ,  $x_5$ , and  $x_6$ ; therefore, we have an infinite number of solutions to this system of equations. A solution is feasible if all of  $x_1, x_2, \dots, x_6$  are nonnegative, and there can be an infinite number of feasible solutions as well. The infinite number of possible solutions to a system such as this one will be useful in later proofs. We focus on the **basic solution**: set all the (nonbasic) variables on the right-hand side to 0 and then compute the values of the (basic) variables on the left-hand side. In this example, the basic solution is  $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_6) = (0, 0, 0, 30, 24, 36)$  and it has objective value  $z = (3 \cdot 0) + (1 \cdot 0) + (2 \cdot 0) = 0$ . Observe that this basic solution sets  $\bar{x}_i = b_i$  for each  $i \in B$ . An iteration of the simplex algorithm rewrites the set of equations and the objective function so as to put a different set of variables on the right-hand side. Thus, a different basic solution is associated with the rewritten problem. We emphasize that the rewrite does not in any way change the underlying linear-programming problem; the problem at one iteration has the identical set of feasible solutions as the problem at the previous iteration. The problem does, however, have a different basic solution than that of the previous iteration.

If a basic solution is also feasible, we call it a **basic feasible solution**. As we run the simplex algorithm, the basic solution is almost always a basic feasible solution. We shall see in Section 29.5, however, that for the first few iterations of the simplex algorithm, the basic solution might not be feasible.

Our goal, in each iteration, is to reformulate the linear program so that the basic solution has a greater objective value. We select a nonbasic variable  $x_e$  whose coefficient in the objective function is positive, and we increase the value of  $x_e$  as much as possible without violating any of the constraints. The variable  $x_e$  becomes basic, and some other variable  $x_l$  becomes nonbasic. The values of other basic variables and of the objective function may also change.

To continue the example, let's think about increasing the value of  $x_1$ . As we increase  $x_1$ , the values of  $x_4$ ,  $x_5$ , and  $x_6$  all decrease. Because we have a nonnegativity constraint for each variable, we cannot allow any of them to become negative. If  $x_1$  increases above 30, then  $x_4$  becomes negative, and  $x_5$  and  $x_6$  become negative when  $x_1$  increases above 12 and 9, respectively. The third constraint (29.61) is the tightest constraint, and it limits how much we can increase  $x_1$ . Therefore, we switch the roles of  $x_1$  and  $x_6$ . We solve equation (29.61) for  $x_1$  and obtain

$$x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4} . \quad (29.62)$$

To rewrite the other equations with  $x_6$  on the right-hand side, we substitute for  $x_1$  using equation (29.62). Doing so for equation (29.59), we obtain

$$\begin{aligned} x_4 &= 30 - x_1 - x_2 - 3x_3 \\ &= 30 - \left(9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4}\right) - x_2 - 3x_3 \\ &= 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4}. \end{aligned} \quad (29.63)$$

Similarly, we combine equation (29.62) with constraint (29.60) and with objective function (29.58) to rewrite our linear program in the following form:

$$z = 27 + \frac{x_2}{4} + \frac{x_3}{2} - \frac{3x_6}{4} \quad (29.64)$$

$$x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4} \quad (29.65)$$

$$x_4 = 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4} \quad (29.66)$$

$$x_5 = 6 - \frac{3x_2}{2} - 4x_3 + \frac{x_6}{2}. \quad (29.67)$$

We call this operation a *pivot*. As demonstrated above, a pivot chooses a nonbasic variable  $x_e$ , called the *entering variable*, and a basic variable  $x_l$ , called the *leaving variable*, and exchanges their roles.

The linear program described in equations (29.64)–(29.67) is equivalent to the linear program described in equations (29.58)–(29.61). We perform two operations in the simplex algorithm: rewrite equations so that variables move between the left-hand side and the right-hand side, and substitute one equation into another. The first operation trivially creates an equivalent problem, and the second, by elementary linear algebra, also creates an equivalent problem. (See Exercise 29.3-3.)

To demonstrate this equivalence, observe that our original basic solution  $(0, 0, 0, 30, 24, 36)$  satisfies the new equations (29.65)–(29.67) and has objective value  $27 + (1/4) \cdot 0 + (1/2) \cdot 0 - (3/4) \cdot 36 = 0$ . The basic solution associated with the new linear program sets the nonbasic values to 0 and is  $(9, 0, 0, 21, 6, 0)$ , with objective value  $z = 27$ . Simple arithmetic verifies that this solution also satisfies equations (29.59)–(29.61) and, when plugged into objective function (29.58), has objective value  $(3 \cdot 9) + (1 \cdot 0) + (2 \cdot 0) = 27$ .

Continuing the example, we wish to find a new variable whose value we wish to increase. We do not want to increase  $x_6$ , since as its value increases, the objective value decreases. We can attempt to increase either  $x_2$  or  $x_3$ ; let us choose  $x_3$ . How far can we increase  $x_3$  without violating any of the constraints? Constraint (29.65) limits it to 18, constraint (29.66) limits it to  $42/5$ , and constraint (29.67) limits it to  $3/2$ . The third constraint is again the tightest one, and therefore we rewrite the third constraint so that  $x_3$  is on the left-hand side and  $x_5$  is on the right-hand

side. We then substitute this new equation,  $x_3 = 3/2 - 3x_2/8 - x_5/4 + x_6/8$ , into equations (29.64)–(29.66) and obtain the new, but equivalent, system

$$z = \frac{111}{4} + \frac{x_2}{16} - \frac{x_5}{8} - \frac{11x_6}{16} \quad (29.68)$$

$$x_1 = \frac{33}{4} - \frac{x_2}{16} + \frac{x_5}{8} - \frac{5x_6}{16} \quad (29.69)$$

$$x_3 = \frac{3}{2} - \frac{3x_2}{8} - \frac{x_5}{4} + \frac{x_6}{8} \quad (29.70)$$

$$x_4 = \frac{69}{4} + \frac{3x_2}{16} + \frac{5x_5}{8} - \frac{x_6}{16} . \quad (29.71)$$

This system has the associated basic solution  $(33/4, 0, 3/2, 69/4, 0, 0)$ , with objective value  $111/4$ . Now the only way to increase the objective value is to increase  $x_2$ . The three constraints give upper bounds of 132, 4, and  $\infty$ , respectively. (We get an upper bound of  $\infty$  from constraint (29.71) because, as we increase  $x_2$ , the value of the basic variable  $x_4$  increases also. This constraint, therefore, places no restriction on how much we can increase  $x_2$ .) We increase  $x_2$  to 4, and it becomes nonbasic. Then we solve equation (29.70) for  $x_2$  and substitute in the other equations to obtain

$$z = 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3} \quad (29.72)$$

$$x_1 = 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3} \quad (29.73)$$

$$x_2 = 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3} \quad (29.74)$$

$$x_4 = 18 - \frac{x_3}{2} + \frac{x_5}{2} . \quad (29.75)$$

At this point, all coefficients in the objective function are negative. As we shall see later in this chapter, this situation occurs only when we have rewritten the linear program so that the basic solution is an optimal solution. Thus, for this problem, the solution  $(8, 4, 0, 18, 0, 0)$ , with objective value 28, is optimal. We can now return to our original linear program given in (29.53)–(29.57). The only variables in the original linear program are  $x_1$ ,  $x_2$ , and  $x_3$ , and so our solution is  $x_1 = 8$ ,  $x_2 = 4$ , and  $x_3 = 0$ , with objective value  $(3 \cdot 8) + (1 \cdot 4) + (2 \cdot 0) = 28$ . Note that the values of the slack variables in the final solution measure how much slack remains in each inequality. Slack variable  $x_4$  is 18, and in inequality (29.54), the left-hand side, with value  $8 + 4 + 0 = 12$ , is 18 less than the right-hand side of 30. Slack variables  $x_5$  and  $x_6$  are 0 and indeed, in inequalities (29.55) and (29.56), the left-hand and right-hand sides are equal. Observe also that even though the coefficients in the original slack form are integral, the coefficients in the other linear programs are not necessarily integral, and the intermediate solutions are not

necessarily integral. Furthermore, the final solution to a linear program need not be integral; it is purely coincidental that this example has an integral solution.

### Pivoting

We now formalize the procedure for pivoting. The procedure PIVOT takes as input a slack form, given by the tuple  $(N, B, A, b, c, v)$ , the index  $l$  of the leaving variable  $x_l$ , and the index  $e$  of the entering variable  $x_e$ . It returns the tuple  $(\hat{N}, \hat{B}, \hat{A}, \hat{b}, \hat{c}, \hat{v})$  describing the new slack form. (Recall again that the entries of the  $m \times n$  matrices  $A$  and  $\hat{A}$  are actually the negatives of the coefficients that appear in the slack form.)

```

PIVOT( $N, B, A, b, c, v, l, e$ )
1  // Compute the coefficients of the equation for new basic variable  $x_e$ .
2  let  $\hat{A}$  be a new  $m \times n$  matrix
3   $\hat{b}_e = b_l / a_{le}$ 
4  for each  $j \in N - \{e\}$ 
5       $\hat{a}_{ej} = a_{lj} / a_{le}$ 
6   $\hat{a}_{el} = 1 / a_{le}$ 
7  // Compute the coefficients of the remaining constraints.
8  for each  $i \in B - \{l\}$ 
9       $\hat{b}_i = b_i - a_{ie} \hat{b}_e$ 
10     for each  $j \in N - \{e\}$ 
11          $\hat{a}_{ij} = a_{ij} - a_{ie} \hat{a}_{ej}$ 
12      $\hat{a}_{il} = -a_{ie} \hat{a}_{el}$ 
13  // Compute the objective function.
14   $\hat{v} = v + c_e \hat{b}_e$ 
15  for each  $j \in N - \{e\}$ 
16       $\hat{c}_j = c_j - c_e \hat{a}_{ej}$ 
17   $\hat{c}_l = -c_e \hat{a}_{el}$ 
18  // Compute new sets of basic and nonbasic variables.
19   $\hat{N} = N - \{e\} \cup \{l\}$ 
20   $\hat{B} = B - \{l\} \cup \{e\}$ 
21  return  $(\hat{N}, \hat{B}, \hat{A}, \hat{b}, \hat{c}, \hat{v})$ 

```

PIVOT works as follows. Lines 3–6 compute the coefficients in the new equation for  $x_e$  by rewriting the equation that has  $x_l$  on the left-hand side to instead have  $x_e$  on the left-hand side. Lines 8–12 update the remaining equations by substituting the right-hand side of this new equation for each occurrence of  $x_e$ . Lines 14–17 do the same substitution for the objective function, and lines 19 and 20 update the

sets of nonbasic and basic variables. Line 21 returns the new slack form. As given, if  $a_{le} = 0$ , PIVOT would cause an error by dividing by 0, but as we shall see in the proofs of Lemmas 29.2 and 29.12, we call PIVOT only when  $a_{le} \neq 0$ .

We now summarize the effect that PIVOT has on the values of the variables in the basic solution.

**Lemma 29.1**

Consider a call to PIVOT( $N, B, A, b, c, v, l, e$ ) in which  $a_{le} \neq 0$ . Let the values returned from the call be  $(\hat{N}, \hat{B}, \hat{A}, \hat{b}, \hat{c}, \hat{v})$ , and let  $\bar{x}$  denote the basic solution after the call. Then

1.  $\bar{x}_j = 0$  for each  $j \in \hat{N}$ .
2.  $\bar{x}_e = b_l / a_{le}$ .
3.  $\bar{x}_i = b_i - a_{ie} \hat{b}_e$  for each  $i \in \hat{B} - \{e\}$ .

**Proof** The first statement is true because the basic solution always sets all nonbasic variables to 0. When we set each nonbasic variable to 0 in a constraint

$$x_i = \hat{b}_i - \sum_{j \in \hat{N}} \hat{a}_{ij} x_j ,$$

we have that  $\bar{x}_i = \hat{b}_i$  for each  $i \in \hat{B}$ . Since  $e \in \hat{B}$ , line 3 of PIVOT gives

$$\bar{x}_e = \hat{b}_e = b_l / a_{le} ,$$

which proves the second statement. Similarly, using line 9 for each  $i \in \hat{B} - \{e\}$ , we have

$$\bar{x}_i = \hat{b}_i = b_i - a_{ie} \hat{b}_e ,$$

which proves the third statement. ■

**The formal simplex algorithm**

We are now ready to formalize the simplex algorithm, which we demonstrated by example. That example was a particularly nice one, and we could have had several other issues to address:

- How do we determine whether a linear program is feasible?
- What do we do if the linear program is feasible, but the initial basic solution is not feasible?
- How do we determine whether a linear program is unbounded?
- How do we choose the entering and leaving variables?

In Section 29.5, we shall show how to determine whether a problem is feasible, and if so, how to find a slack form in which the initial basic solution is feasible. Therefore, let us assume that we have a procedure INITIALIZE-SIMPLEX( $A, b, c$ ) that takes as input a linear program in standard form, that is, an  $m \times n$  matrix  $A = (a_{ij})$ , an  $m$ -vector  $b = (b_i)$ , and an  $n$ -vector  $c = (c_j)$ . If the problem is infeasible, the procedure returns a message that the program is infeasible and then terminates. Otherwise, the procedure returns a slack form for which the initial basic solution is feasible.

The procedure SIMPLEX takes as input a linear program in standard form, as just described. It returns an  $n$ -vector  $\bar{x} = (\bar{x}_j)$  that is an optimal solution to the linear program described in (29.19)–(29.21).

SIMPLEX( $A, b, c$ )

```

1  ( $N, B, A, b, c, v$ ) = INITIALIZE-SIMPLEX( $A, b, c$ )
2  let  $\Delta$  be a new vector of length  $m$ 
3  while some index  $j \in N$  has  $c_j > 0$ 
4      choose an index  $e \in N$  for which  $c_e > 0$ 
5      for each index  $i \in B$ 
6          if  $a_{ie} > 0$ 
7               $\Delta_i = b_i / a_{ie}$ 
8          else  $\Delta_i = \infty$ 
9      choose an index  $l \in B$  that minimizes  $\Delta_l$ 
10     if  $\Delta_l == \infty$ 
11         return “unbounded”
12     else ( $N, B, A, b, c, v$ ) = PIVOT( $N, B, A, b, c, v, l, e$ )
13 for  $i = 1$  to  $n$ 
14     if  $i \in B$ 
15          $\bar{x}_i = b_i$ 
16     else  $\bar{x}_i = 0$ 
17 return ( $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n$ )

```

The SIMPLEX procedure works as follows. In line 1, it calls the procedure INITIALIZE-SIMPLEX( $A, b, c$ ), described above, which either determines that the linear program is infeasible or returns a slack form for which the basic solution is feasible. The **while** loop of lines 3–12 forms the main part of the algorithm. If all coefficients in the objective function are negative, then the **while** loop terminates. Otherwise, line 4 selects a variable  $x_e$ , whose coefficient in the objective function is positive, as the entering variable. Although we may choose any such variable as the entering variable, we assume that we use some prespecified deterministic rule. Next, lines 5–9 check each constraint and pick the one that most severely limits the amount by which we can increase  $x_e$  without violating any of the nonnegativ-

ity constraints; the basic variable associated with this constraint is  $x_l$ . Again, we are free to choose one of several variables as the leaving variable, but we assume that we use some prespecified deterministic rule. If none of the constraints limits the amount by which the entering variable can increase, the algorithm returns “unbounded” in line 11. Otherwise, line 12 exchanges the roles of the entering and leaving variables by calling  $\text{PIVOT}(N, B, A, b, c, v, l, e)$ , as described above. Lines 13–16 compute a solution  $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n$  for the original linear-programming variables by setting all the nonbasic variables to 0 and each basic variable  $\bar{x}_i$  to  $b_i$ , and line 17 returns these values.

To show that **SIMPLEX** is correct, we first show that if **SIMPLEX** has an initial feasible solution and eventually terminates, then it either returns a feasible solution or determines that the linear program is unbounded. Then, we show that **SIMPLEX** terminates. Finally, in Section 29.4 (Theorem 29.10) we show that the solution returned is optimal.

### **Lemma 29.2**

Given a linear program  $(A, b, c)$ , suppose that the call to **INITIALIZE-SIMPLEX** in line 1 of **SIMPLEX** returns a slack form for which the basic solution is feasible. Then if **SIMPLEX** returns a solution in line 17, that solution is a feasible solution to the linear program. If **SIMPLEX** returns “unbounded” in line 11, the linear program is unbounded.

**Proof** We use the following three-part loop invariant:

At the start of each iteration of the **while** loop of lines 3–12,

1. the slack form is equivalent to the slack form returned by the call of **INITIALIZE-SIMPLEX**,
2. for each  $i \in B$ , we have  $b_i \geq 0$ , and
3. the basic solution associated with the slack form is feasible.

**Initialization:** The equivalence of the slack forms is trivial for the first iteration. We assume, in the statement of the lemma, that the call to **INITIALIZE-SIMPLEX** in line 1 of **SIMPLEX** returns a slack form for which the basic solution is feasible. Thus, the third part of the invariant is true. Because the basic solution is feasible, each basic variable  $x_i$  is nonnegative. Furthermore, since the basic solution sets each basic variable  $x_i$  to  $b_i$ , we have that  $b_i \geq 0$  for all  $i \in B$ . Thus, the second part of the invariant holds.

**Maintenance:** We shall show that each iteration of the **while** loop maintains the loop invariant, assuming that the **return** statement in line 11 does not execute. We shall handle the case in which line 11 executes when we discuss termination.

An iteration of the **while** loop exchanges the role of a basic and a nonbasic variable by calling the PIVOT procedure. By Exercise 29.3-3, the slack form is equivalent to the one from the previous iteration which, by the loop invariant, is equivalent to the initial slack form.

We now demonstrate the second part of the loop invariant. We assume that at the start of each iteration of the **while** loop,  $b_i \geq 0$  for each  $i \in B$ , and we shall show that these inequalities remain true after the call to PIVOT in line 12. Since the only changes to the variables  $b_i$  and the set  $B$  of basic variables occur in this assignment, it suffices to show that line 12 maintains this part of the invariant. We let  $b_i$ ,  $a_{ij}$ , and  $B$  refer to values before the call of PIVOT, and  $\hat{b}_i$  refer to values returned from PIVOT.

First, we observe that  $\hat{b}_e \geq 0$  because  $b_l \geq 0$  by the loop invariant,  $a_{le} > 0$  by lines 6 and 9 of SIMPLEX, and  $\hat{b}_e = b_l/a_{le}$  by line 3 of PIVOT.

For the remaining indices  $i \in B - \{l\}$ , we have that

$$\begin{aligned}\hat{b}_i &= b_i - a_{ie}\hat{b}_e && \text{(by line 9 of PIVOT)} \\ &= b_i - a_{ie}(b_l/a_{le}) && \text{(by line 3 of PIVOT)} .\end{aligned}\tag{29.76}$$

We have two cases to consider, depending on whether  $a_{ie} > 0$  or  $a_{ie} \leq 0$ . If  $a_{ie} > 0$ , then since we chose  $l$  such that

$$b_l/a_{le} \leq b_i/a_{ie} \quad \text{for all } i \in B ,\tag{29.77}$$

we have

$$\begin{aligned}\hat{b}_i &= b_i - a_{ie}(b_l/a_{le}) && \text{(by equation (29.76))} \\ &\geq b_i - a_{ie}(b_i/a_{ie}) && \text{(by inequality (29.77))} \\ &= b_i - b_i \\ &= 0 ,\end{aligned}$$

and thus  $\hat{b}_i \geq 0$ . If  $a_{ie} \leq 0$ , then because  $a_{le}$ ,  $b_i$ , and  $b_l$  are all nonnegative, equation (29.76) implies that  $\hat{b}_i$  must be nonnegative, too.

We now argue that the basic solution is feasible, i.e., that all variables have nonnegative values. The nonbasic variables are set to 0 and thus are nonnegative. Each basic variable  $x_i$  is defined by the equation

$$x_i = b_i - \sum_{j \in N} a_{ij}x_j .$$

The basic solution sets  $\bar{x}_i = b_i$ . Using the second part of the loop invariant, we conclude that each basic variable  $\bar{x}_i$  is nonnegative.



**Termination:** The **while** loop can terminate in one of two ways. If it terminates because of the condition in line 3, then the current basic solution is feasible and line 17 returns this solution. The other way it terminates is by returning “unbounded” in line 11. In this case, for each iteration of the **for** loop in lines 5–8, when line 6 is executed, we find that  $a_{ie} \leq 0$ . Consider the solution  $\bar{x}$  defined as

$$\bar{x}_i = \begin{cases} \infty & \text{if } i = e, \\ 0 & \text{if } i \in N - \{e\}, \\ b_i - \sum_{j \in N} a_{ij} \bar{x}_j & \text{if } i \in B. \end{cases}$$

We now show that this solution is feasible, i.e., that all variables are nonnegative. The nonbasic variables other than  $\bar{x}_e$  are 0, and  $\bar{x}_e = \infty > 0$ ; thus all nonbasic variables are nonnegative. For each basic variable  $\bar{x}_i$ , we have

$$\begin{aligned} \bar{x}_i &= b_i - \sum_{j \in N} a_{ij} \bar{x}_j \\ &= b_i - a_{ie} \bar{x}_e. \end{aligned}$$

The loop invariant implies that  $b_i \geq 0$ , and we have  $a_{ie} \leq 0$  and  $\bar{x}_e = \infty > 0$ . Thus,  $\bar{x}_i \geq 0$ .

Now we show that the objective value for the solution  $\bar{x}$  is unbounded. From equation (29.42), the objective value is

$$\begin{aligned} z &= v + \sum_{j \in N} c_j \bar{x}_j \\ &= v + c_e \bar{x}_e. \end{aligned}$$

Since  $c_e > 0$  (by line 4 of SIMPLEX) and  $\bar{x}_e = \infty$ , the objective value is  $\infty$ , and thus the linear program is unbounded. ■

It remains to show that SIMPLEX terminates, and when it does terminate, the solution it returns is optimal. Section 29.4 will address optimality. We now discuss termination.

### Termination

In the example given in the beginning of this section, each iteration of the simplex algorithm increased the objective value associated with the basic solution. As Exercise 29.3-2 asks you to show, no iteration of SIMPLEX can decrease the objective value associated with the basic solution. Unfortunately, it is possible that an iteration leaves the objective value unchanged. This phenomenon is called *degeneracy*, and we shall now study it in greater detail.

The assignment in line 14 of PIVOT,  $\hat{v} = v + c_e \hat{b}_e$ , changes the objective value. Since SIMPLEX calls PIVOT only when  $c_e > 0$ , the only way for the objective value to remain unchanged (i.e.,  $\hat{v} = v$ ) is for  $\hat{b}_e$  to be 0. This value is assigned as  $\hat{b}_e = b_l / a_{le}$  in line 3 of PIVOT. Since we always call PIVOT with  $a_{le} \neq 0$ , we see that for  $\hat{b}_e$  to equal 0, and hence the objective value to be unchanged, we must have  $b_l = 0$ .

Indeed, this situation can occur. Consider the linear program

$$\begin{aligned} z &= & x_1 &+& x_2 &+& x_3 \\ x_4 &= 8 &-& x_1 &-& x_2 \\ x_5 &= &&& x_2 &-& x_3 . \end{aligned}$$

Suppose that we choose  $x_1$  as the entering variable and  $x_4$  as the leaving variable. After pivoting, we obtain

$$\begin{aligned} z &= 8 &&& +& x_3 &-& x_4 \\ x_1 &= 8 &-& x_2 &&& -& x_4 \\ x_5 &= && x_2 &-& x_3 . \end{aligned}$$

At this point, our only choice is to pivot with  $x_3$  entering and  $x_5$  leaving. Since  $b_5 = 0$ , the objective value of 8 remains unchanged after pivoting:

$$\begin{aligned} z &= 8 &+& x_2 &-& x_4 &-& x_5 \\ x_1 &= 8 &-& x_2 &-& x_4 \\ x_3 &= && x_2 &&& -& x_5 . \end{aligned}$$

The objective value has not changed, but our slack form has. Fortunately, if we pivot again, with  $x_2$  entering and  $x_1$  leaving, the objective value increases (to 16), and the simplex algorithm can continue.

Degeneracy can prevent the simplex algorithm from terminating, because it can lead to a phenomenon known as **cycling**: the slack forms at two different iterations of SIMPLEX are identical. Because of degeneracy, SIMPLEX could choose a sequence of pivot operations that leave the objective value unchanged but repeat a slack form within the sequence. Since SIMPLEX is a deterministic algorithm, if it cycles, then it will cycle through the same series of slack forms forever, never terminating.

Cycling is the only reason that SIMPLEX might not terminate. To show this fact, we must first develop some additional machinery.

At each iteration, SIMPLEX maintains  $A$ ,  $b$ ,  $c$ , and  $v$  in addition to the sets  $N$  and  $B$ . Although we need to explicitly maintain  $A$ ,  $b$ ,  $c$ , and  $v$  in order to implement the simplex algorithm efficiently, we can get by without maintaining them. In other words, the sets of basic and nonbasic variables suffice to uniquely determine the slack form. Before proving this fact, we prove a useful algebraic lemma.

**Lemma 29.3**

Let  $I$  be a set of indices. For each  $j \in I$ , let  $\alpha_j$  and  $\beta_j$  be real numbers, and let  $x_j$  be a real-valued variable. Let  $\gamma$  be any real number. Suppose that for any settings of the  $x_j$ , we have

$$\sum_{j \in I} \alpha_j x_j = \gamma + \sum_{j \in I} \beta_j x_j . \quad (29.78)$$

Then  $\alpha_j = \beta_j$  for each  $j \in I$ , and  $\gamma = 0$ .

**Proof** Since equation (29.78) holds for any values of the  $x_j$ , we can use particular values to draw conclusions about  $\alpha$ ,  $\beta$ , and  $\gamma$ . If we let  $x_j = 0$  for each  $j \in I$ , we conclude that  $\gamma = 0$ . Now pick an arbitrary index  $j \in I$ , and set  $x_j = 1$  and  $x_k = 0$  for all  $k \neq j$ . Then we must have  $\alpha_j = \beta_j$ . Since we picked  $j$  as any index in  $I$ , we conclude that  $\alpha_j = \beta_j$  for each  $j \in I$ . ■

A particular linear program has many different slack forms; recall that each slack form has the same set of feasible and optimal solutions as the original linear program. We now show that the slack form of a linear program is uniquely determined by the set of basic variables. That is, given the set of basic variables, a unique slack form (unique set of coefficients and right-hand sides) is associated with those basic variables.

**Lemma 29.4**

Let  $(A, b, c)$  be a linear program in standard form. Given a set  $B$  of basic variables, the associated slack form is uniquely determined.

**Proof** Assume for the purpose of contradiction that there are two different slack forms with the same set  $B$  of basic variables. The slack forms must also have identical sets  $N = \{1, 2, \dots, n + m\} - B$  of nonbasic variables. We write the first slack form as

$$z = v + \sum_{j \in N} c_j x_j \quad (29.79)$$

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j \text{ for } i \in B , \quad (29.80)$$

and the second as

$$z = v' + \sum_{j \in N} c'_j x_j \quad (29.81)$$

$$x_i = b'_i - \sum_{j \in N} a'_{ij} x_j \text{ for } i \in B . \quad (29.82)$$

Consider the system of equations formed by subtracting each equation in line (29.82) from the corresponding equation in line (29.80). The resulting system is

$$0 = (b_i - b'_i) - \sum_{j \in N} (a_{ij} - a'_{ij})x_j \quad \text{for } i \in B$$

or, equivalently,

$$\sum_{j \in N} a_{ij}x_j = (b_i - b'_i) + \sum_{j \in N} a'_{ij}x_j \quad \text{for } i \in B.$$

Now, for each  $i \in B$ , apply Lemma 29.3 with  $\alpha_j = a_{ij}$ ,  $\beta_j = a'_{ij}$ ,  $\gamma = b_i - b'_i$ , and  $I = N$ . Since  $\alpha_j = \beta_j$ , we have that  $a_{ij} = a'_{ij}$  for each  $j \in N$ , and since  $\gamma = 0$ , we have that  $b_i = b'_i$ . Thus, for the two slack forms,  $A$  and  $b$  are identical to  $A'$  and  $b'$ . Using a similar argument, Exercise 29.3-1 shows that it must also be the case that  $c = c'$  and  $v = v'$ , and hence that the slack forms must be identical. ■

We can now show that cycling is the only possible reason that SIMPLEX might not terminate.

### Lemma 29.5

If SIMPLEX fails to terminate in at most  $\binom{n+m}{m}$  iterations, then it cycles.

**Proof** By Lemma 29.4, the set  $B$  of basic variables uniquely determines a slack form. There are  $n + m$  variables and  $|B| = m$ , and therefore, there are at most  $\binom{n+m}{m}$  ways to choose  $B$ . Thus, there are only at most  $\binom{n+m}{m}$  unique slack forms. Therefore, if SIMPLEX runs for more than  $\binom{n+m}{m}$  iterations, it must cycle. ■

Cycling is theoretically possible, but extremely rare. We can prevent it by choosing the entering and leaving variables somewhat more carefully. One option is to perturb the input slightly so that it is impossible to have two solutions with the same objective value. Another option is to break ties by always choosing the variable with the smallest index, a strategy known as **Bland's rule**. We omit the proof that these strategies avoid cycling.

### Lemma 29.6

If lines 4 and 9 of SIMPLEX always break ties by choosing the variable with the smallest index, then SIMPLEX must terminate. ■

We conclude this section with the following lemma.

**Lemma 29.7**

Assuming that INITIALIZE-SIMPLEX returns a slack form for which the basic solution is feasible, SIMPLEX either reports that a linear program is unbounded, or it terminates with a feasible solution in at most  $\binom{n+m}{m}$  iterations.

**Proof** Lemmas 29.2 and 29.6 show that if INITIALIZE-SIMPLEX returns a slack form for which the basic solution is feasible, SIMPLEX either reports that a linear program is unbounded, or it terminates with a feasible solution. By the contrapositive of Lemma 29.5, if SIMPLEX terminates with a feasible solution, then it terminates in at most  $\binom{n+m}{m}$  iterations. ■

**Exercises****29.3-1**

Complete the proof of Lemma 29.4 by showing that it must be the case that  $c = c'$  and  $v = v'$ .

**29.3-2**

Show that the call to PIVOT in line 12 of SIMPLEX never decreases the value of  $v$ .

**29.3-3**

Prove that the slack form given to the PIVOT procedure and the slack form that the procedure returns are equivalent.

**29.3-4**

Suppose we convert a linear program  $(A, b, c)$  in standard form to slack form. Show that the basic solution is feasible if and only if  $b_i \geq 0$  for  $i = 1, 2, \dots, m$ .

**29.3-5**

Solve the following linear program using SIMPLEX:

$$\begin{array}{llll}
 \text{maximize} & 18x_1 & + & 12.5x_2 \\
 \text{subject to} & & & \\
 & x_1 & + & x_2 \leq 20 \\
 & x_1 & & \leq 12 \\
 & & x_2 & \leq 16 \\
 & x_1, x_2 & & \geq 0 .
 \end{array}$$

**29.3-6**

Solve the following linear program using SIMPLEX:

$$\begin{array}{llll}
 \text{maximize} & 5x_1 & - & 3x_2 \\
 \text{subject to} & & & \\
 & x_1 & - & x_2 \leq 1 \\
 & 2x_1 & + & x_2 \leq 2 \\
 & x_1, x_2 & & \geq 0 .
 \end{array}$$

**29.3-7**

Solve the following linear program using SIMPLEX:

$$\begin{array}{llllll}
 \text{minimize} & x_1 & + & x_2 & + & x_3 \\
 \text{subject to} & & & & & \\
 & 2x_1 & + & 7.5x_2 & + & 3x_3 \geq 10000 \\
 & 20x_1 & + & 5x_2 & + & 10x_3 \geq 30000 \\
 & x_1, x_2, x_3 & & & & \geq 0 .
 \end{array}$$

**29.3-8**

In the proof of Lemma 29.5, we argued that there are at most  $\binom{m+n}{n}$  ways to choose a set  $B$  of basic variables. Give an example of a linear program in which there are strictly fewer than  $\binom{m+n}{n}$  ways to choose the set  $B$ .

---

**29.4 Duality**

We have proven that, under certain assumptions, SIMPLEX terminates. We have not yet shown that it actually finds an optimal solution to a linear program, however. In order to do so, we introduce a powerful concept called **linear-programming duality**.

Duality enables us to prove that a solution is indeed optimal. We saw an example of duality in Chapter 26 with Theorem 26.6, the max-flow min-cut theorem. Suppose that, given an instance of a maximum-flow problem, we find a flow  $f$  with value  $|f|$ . How do we know whether  $f$  is a maximum flow? By the max-flow min-cut theorem, if we can find a cut whose value is also  $|f|$ , then we have verified that  $f$  is indeed a maximum flow. This relationship provides an example of duality: given a maximization problem, we define a related minimization problem such that the two problems have the same optimal objective values.

Given a linear program in which the objective is to maximize, we shall describe how to formulate a **dual** linear program in which the objective is to minimize and